



# 实验三：深度学习模型优化 实践

《深度学习》实验课  
2026年 春

# 目录

---

|   |           |
|---|-----------|
| 1 | 常见的模型优化方法 |
| 2 | 主流优化器介绍   |
| 3 | 案例应用      |
| 4 | 实验作业      |

# 常见的模型优化方法

---

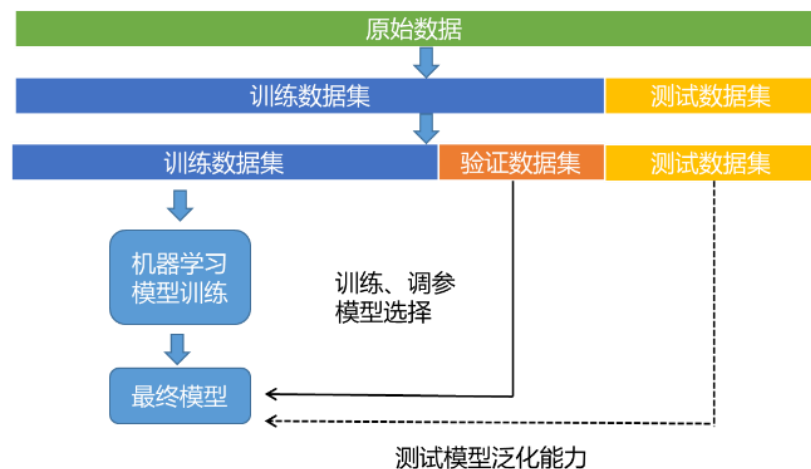
-  常见的模型优化方法有以下几种：
- 1、防止模型过拟合（正则化方法）
  - 2、数据拆分：拆分出训练集、验证集（K折交叉验证）、测试集
  - 3、多参数调优（网格搜索方法）
  - 4、尝试多种优化器（基于梯度的优化，本次实验重点讲解）
  - 5、设计复杂的网络模型（CNN、DNN等等）（下次实验）

# 常见的模型优化方法



## 数据拆分：训练集、验证集、测试集

- 在模型的训练过程中可以引入验证集策略来防止模型的过拟合，即将数据集分为3个子集：训练集，用来训练模型；验证集，用来验证模型效果，帮助模型调优；测试集，用来测试模型的泛化能力，避免模型过拟合。



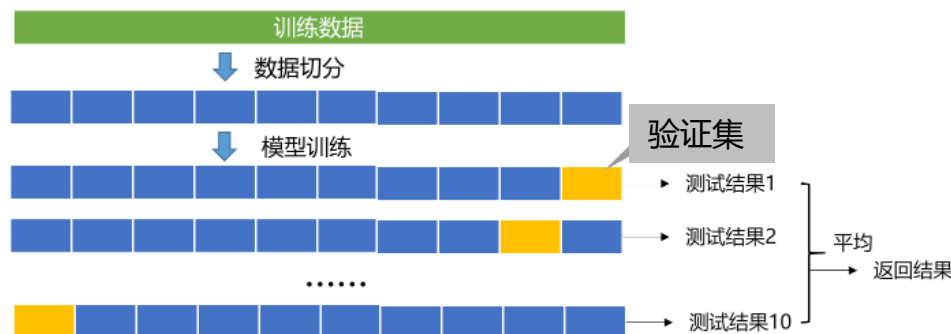
- Scikit-Learn中 `model_selection`子模块的**`train_test_split()`**函数可以将数据集随机切分为训练集和测试集，其主要参数描述如下：
  - ❑ `test_size`: 如果为float类型，则应在0.0到1.0之间，表示要包含在训练集切分中的数据集的比例。如果为int类型，则表示训练集样本的绝对数量。
  - ❑ `random_state`: 随机数种子，确保每次运行可以得到一样的结果。
  - ❑ `shuffle`: 是否重新洗牌，默认为True。即将你的数据集打乱，重新排列。
  - ❑ `stratify`: 按照一定的比例抽取样本。

# 常见的模型优化方法



## 数据拆分: K折交叉验证 → 找出最优的超参数

- K折交叉验证是采用某种方式将**训练集**切分为k个子集，每次采用其中的一个子集作为模型的**验证集**，余下的k-1个子集用于模型训练；这个过程重复k次，每次选取作为验证集均不相同，直到每个子集都测试过；最终将k次测试结果的均值作为模型的效果评价。



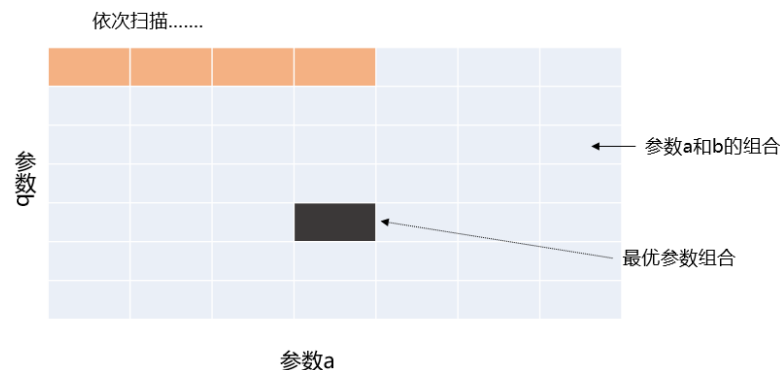
- Scikit-Learn中model\_selection子模块的**cross\_val\_score()**函数可以实现k折交叉验证法，主要参数如下：
  - ❑ estimator: 需要评估的学习器。
  - ❑ X: 数组类型数据，训练集（自变量部分）。
  - ❑ y: 数组类型数据，训练集（因变量部分），可选。
  - ❑ cv: int类型，要将数据集分割的折数，默认的情况下是5。
  - ❑ fit\_params: 一个字典，传递给学习器拟合的方法。
  - ❑ scoring: 评估学习器性能的标准，通常需要使用 `sklearn.metrics` 中的函数，如 “Accuracy”，“mean\_absolute\_error”，“mean\_squared\_error”等。

# 常见的模型优化方法



## 多参数调优（网格搜索方法）

- 网格搜索的基本原理是将各参数变量值的区间划分为一系列的小区间，并按顺序计算出对应各参数变量值组合所确定的目标值（通常是误差），并逐一择优，以得到该区间内最小目标值及其对应的最佳参数值。该方法可保证所得的搜索解释**全局最优或接近最优**的，可避免产生重大的误差。



- Scikit-Learn中的model\_selection子模块中**GridSearchCV()**方法可以实现网格搜索，主要参数描述如下：
  - ❑ estimator: 选择使用的分类器，并且传入除需要确定最佳的参数之外的其他参数。每一个分类器都需要一个scoring参数，或者score方法。
  - ❑ param\_grid: 需要最优化的参数的取值，值为字典或者列表。
  - ❑ scoring = None: 模型评价标准，默认为None，这时需要使用score函数；或者如scoring = 'roc\_auc'，根据所选模型不同，评价准则不同。
  - ❑ cv = None: 交叉验证参数，默认None，使用五折交叉验证。指定fold数量，默认为5，也可以是yield训练/测试数据的生成器。

注意：**GridSearchCV()**函数在深度学习**PyTorch**框架下使用可能会有一些**兼容性问题**，不能直接使用，我们需要手动实现类似的逻辑。

# 目录

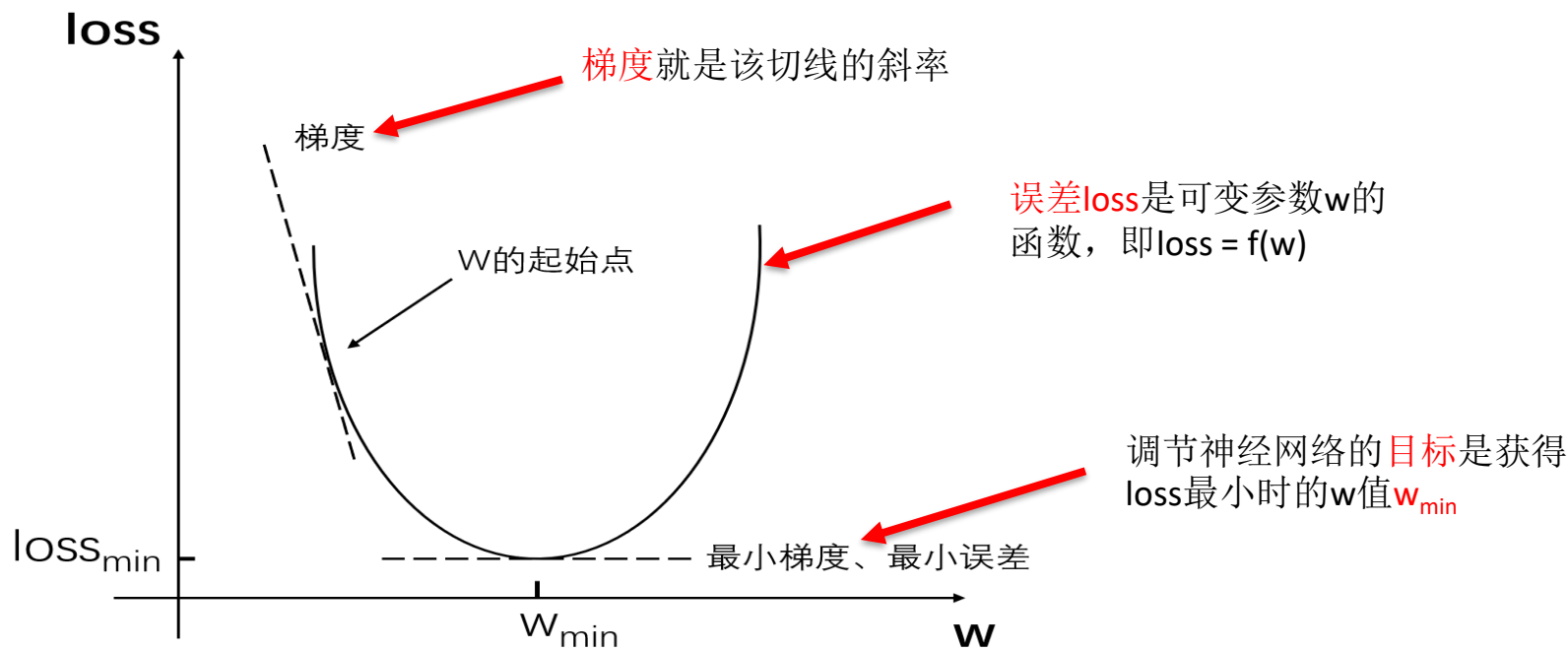
---

|   |           |
|---|-----------|
| 1 | 常见的模型优化方法 |
| 2 | 主流优化器介绍   |
| 3 | 案例应用      |
| 4 | 实验作业      |

# 优化器的作用



优化器在神经网络的训练中具有相当重要的地位，它的主要作用是根据误差函数的计算结果来调节可变参数，使得神经网络的输出越来越符合预期。梯度下降算法是神经网络中流行的优化算法之一，它能够很好地解决一系列问题。

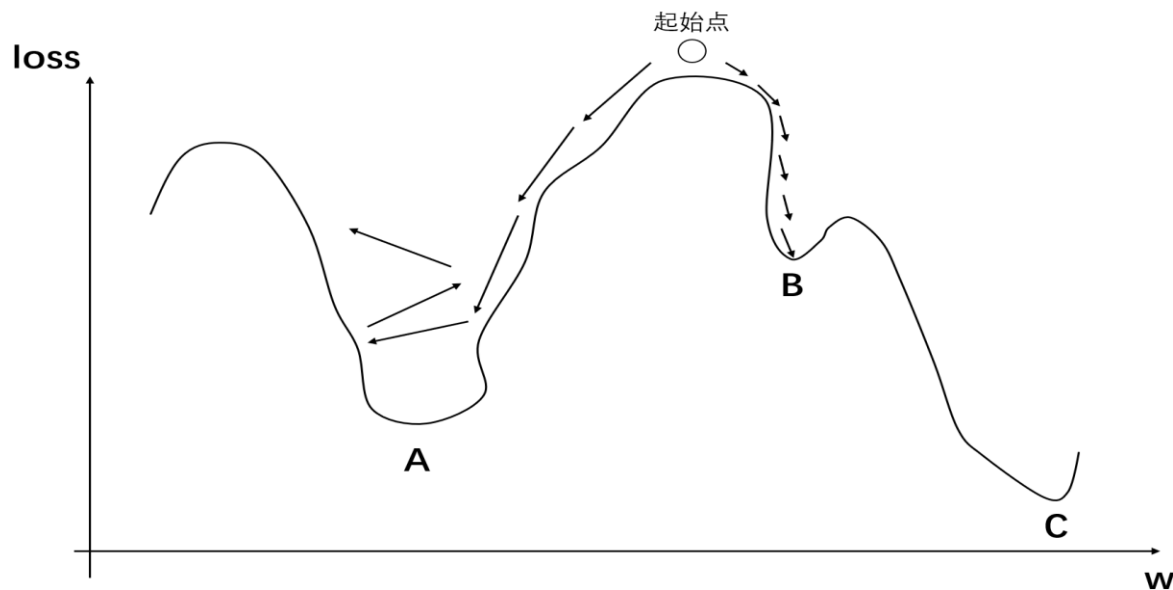


- 梯度下降算法迭代地更新参数，以使整体网络的误差最小化。

# 优化器的作用



## 学习率对训练的影响



- **学习率**控制的是调节可变参数的幅度，是优化器的一个参数设置项。形象地说，**学习率**代表下山的**步幅**，越大越快，但容易走过头。
- **调整学习率**主要有两种做法。一是修改优化器内的对应学习率，二是新建优化器。在训练之前，我们不清楚一个特定问题设置成怎样的学习率是合理的，因此在训练时可以尝试调小或调大，通过观察loss的下降情况判断合理的学习率。

# 主流优化器介绍



- **随机梯度下降**（Stochastic Gradient Descent，简称**SGD**）的方法则是仅随机使用一组数据来进行梯度计算，也就是下山过程中每次走的一步带有一定的随机性，这样可以比普通梯度下降算法**节省很多时间**，但缺陷是有更大的可能陷入**局部最优解**（即到达非最低的谷底）。
- **Adagrad算法**是一种自适应学习率的优化算法，它可以自适应地为每个参数分配不同的学习率，随着梯度调整的过程，学习率会逐步下降，这样可以避免一些学习率过大导致的跳过最优解的情况。同时，Adagrad算法对可变参数会自动根据一定的规则来使用**不同的调整幅度**。因此，对于**稀疏的数据**它的表现很好，能很好地提高了随机梯度下降法（SGD）的鲁棒性。
- **Adadelta**和**RMSProp**这两种算法非常类似，都是对Adagrad算法的一个改进，它们试图解决Adagrad算法中**对学习率过于激进而单调的不断减少**带来的问题。RMSProp算法采用了指数衰减的方式淡化历史对当前步骤参数更新量的影响。
- **Adam**（Adaptive Moment Estimation）是一种自适应学习率的优化算法，组合了动量法（Momentum）和RMSProp的优化算法，通过Momentum加速收敛，并通过学习率衰减自动调整学习率。Adam算法这结合了两类算法的优点，是**对每一个可变参数都计算动态学习率**的算法，也是目前**首要推荐**的算法。

# 优化器的对比



在选择优化器时，应该考虑以下几个因素：

- 1、数据集的规模和复杂性

对于大规模数据集和复杂模型，应该选择具有较好收敛性的优化器，如 SGD 或 Adam。

- 2、模型的稀疏性

对于稀疏模型，应该选择 Adagrad 或其他自适应学习率的优化器。

- 3、训练速度

对于需要较快训练速度的场景，可以选择具有加速功能的优化器，如 SGD 的带动量更新或 Nesterov 动量更新，或者选择具有较高并行性的优化器，如 Adam 或 Adagrad。

- 4、学习率的调整

对于需要调整学习率的场景，可以选择自适应学习率的优化器，如 Adam 或 Adagrad。在实际使用中，可以通过尝试多种优化器并比较其效果来选择最合适的优化器。

总的来说，Adam是首选，RMSProp和Adadelata可以作为备选，普通梯度下降算法相对最慢。

# 优化器的代码编程

 **optimizer = optim.优化器名称( model.parameters(), 其他参数 )**

- torch.optim方法中内置了各种优化器，每种优化器都可以自己设置学习率、动量和超参数、以及其他可选项，具体示例应用如下。

| 优化器  | 使用方法                                                                                                                                                                                                                                                                                                                                                                               |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SGD  | <pre># 定义SGD优化器[citation:1][citation:9] optimizer = optim.SGD(     model.parameters(), # 要优化的模型参数     lr=0.01,             # 学习率     momentum=0.9,        # 动量因子     weight_decay=0.001,  # 权重衰减 (L2正则化)     nesterov=False       # 是否使用Nesterov动量 )</pre>                                                                                                                         |
| Adam | <pre># 定义Adam优化器[citation:7] optimizer = optim.Adam(     model.parameters(), # 要优化的模型参数[citation:2]     lr=0.001,           # 学习率[citation:7]     betas=(0.9, 0.999), # 用于计算梯度以及梯度平方的运行平均值的系数[citation:8]     eps=1e-08,          # 为了增加数值稳定性而添加到分母的项[citation:8]     weight_decay=0,     # 权重衰减 (L2惩罚) [citation:8]     amsgrad=False       # 是否使用AMSGrad变种算法[citation:8] )</pre> |

# 优化器的代码编程

| 优化器      | 使用方法                                                                                                                                                                                                                                                                                                                                           |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Adagrad  | <pre># 定义Adagrad优化器 optimizer = optim.Adagrad(     model.parameters(), # 要优化的模型参数     lr=0.01,             # 学习率     lr_decay=0,          # 学习率衰减     weight_decay=0,      # 权重衰减 (L2正则化)     initial_accumulator_value=0, # 梯度累加器的初始值     eps=1e-10            # 防止除零的小常数 )</pre>                                                             |
| RMSprop  | <pre># 定义RMSprop优化器 optimizer = optim.RMSprop(     model.parameters(), # 要优化的模型参数     lr=0.01,            # 学习率     alpha=0.99,         # 平滑常数, 控制历史梯度的影响[citation:9]     eps=1e-08,          # 防止除零的小常数[citation:9]     weight_decay=0,      # 权重衰减 (L2正则化)     momentum=0,          # 动量因子     centered=False       # 是否使用中心化RMSprop )</pre> |
| Adadelta | <pre># Adadelta优化器 optimizer = optim.Adadelta(     model.parameters(),     lr=0.1,             # 学习率     rho=0.95,           # 衰减率     eps=1e-5,           # 数值稳定性项     weight_decay=1e-4   # L2正则化 )</pre>                                                                                                                                    |



除了选择合适的优化器之外，优化器的学习率参数是最不可忽视的因素，当训练结果始终无法提升的时候，可以适当调节学习率试试。

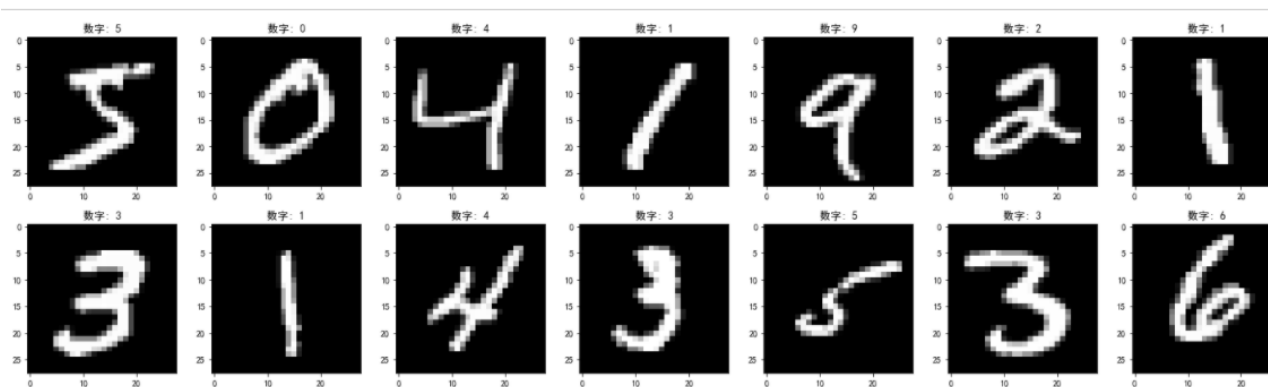
# 目录

---

|   |           |
|---|-----------|
| 1 | 常见的模型优化方法 |
| 2 | 主流优化器介绍   |
| 3 | 案例应用      |
| 4 | 实验作业      |

# MNIST数据集概述

- **MNIST数据集**共有**训练数据60000项**、**测试数据10000项**。MNIST数据集中的每一项数据都由images（数字图像）和labels（真实的数字）所组成。MNIST数据集预先加载在tf.keras中，其中包含4个Numpy数组。
- **MNIST数据集**下载地址：<http://yann.lecun.com/exdb/mnist/>，从官方网站下载的数据是gz格式的压缩包，解压后可以得到原始文件。mnist数据集包含**4个文件**，分别对应60000个训练图片，60000个训练标签，10000个测试图片，10000个测试标签。
  - train-images-idx3-ubyte.gz: 60000个训练图片 (9912422 bytes)
  - train-labels-idx1-ubyte.gz: 60000个训练标签 (28881 bytes)
  - t10k-images-idx3-ubyte.gz: 10000个测试图片 (1648877 bytes)
  - t10k-labels-idx1-ubyte.gz: 10000个测试标签 (4542 bytes)



- **MNIST显示**——在MNIST数据集中的每一张图片都代表了0~9中的一个数字。图片的大小都为28\*28，且数字都会出现在图片的正中间。

# 1、数据导入



- 首先导入MNIST数据集，划分训练集和测试集数据。

```
# 导入必要的库
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# 设置设备 - 检查是否有可用的GPU，如果有则使用GPU，否则使用CPU
# 这可以显著加速训练过程，因为GPU擅长并行计算
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"使用设备: {device}") # 打印当前使用的设备信息
```

```
# 数据预处理和加载
# transforms.Compose 将多个数据转换操作组合在一起
transform = transforms.Compose([
    transforms.ToTensor(), # 将PIL图像或numpy数组转换为PyTorch张量
    transforms.Normalize((0.1307,), (0.3081,)) # 使用MNIST数据集的均值和标准差进行标准化
    # 标准化公式: (input - mean) / std
])

# 加载MNIST数据集
# datasets.MNIST是PyTorch提供的MNIST数据集加载器
# root: 数据集存储路径
# train=True: 加载训练集
# download=True: 如果数据集不存在则自动下载
# transform: 应用上面定义的数据转换
train_dataset = datasets.MNIST('./data', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST('./data', train=False, download=True, transform=transform)
```

注意：MNIST数据集能直接导出区分训练集和测试集，没有用到数据划分；而实验作业需要用到`train_test_split()`函数划分数据集。

## 2、带交叉验证思想的网格搜索验证（1）



- 配置不同优化器参数组合

```
# 不同优化器的参数组合
params_list = [
    # Adam优化器
    {'optimizer': 'Adam', 'lr': 0.001, 'hidden_size': 128},
    {'optimizer': 'Adam', 'lr': 0.01, 'hidden_size': 128},

    # Adagrad优化器
    {'optimizer': 'Adagrad', 'lr': 0.1, 'hidden_size': 128},
    {'optimizer': 'Adagrad', 'lr': 0.01, 'hidden_size': 256},
]
```

可以设置优化器类型，及对应的参数，以及神经网络的隐藏层数等等参数，集成在一个python 列表/字典里。

- 开启K折交叉验证，划分训练集和验证集。

```
best_score = 0
best_params = None
K = 3

for params in params_list:
    print(f"\n测试参数: {params}")

    scores = []

    # K折交叉验证
    for fold in range(K):
        # 分割数据
        n = len(train_dataset)
        fold_size = n // K
        indices = list(range(n))
        np.random.shuffle(indices)

        # 计算当前折的训练集和验证集索引
        val_start = fold * fold_size
        val_end = val_start + fold_size
        if fold == K-1:
            val_end = n
        val_idx = indices[val_start:val_end]
        train_idx = indices[:val_start] + indices[val_end:]

        # 创建数据加载器
        train_subset = torch.utils.data.Subset(train_dataset, train_idx)
        val_subset = torch.utils.data.Subset(train_dataset, val_idx)
        train_loader_fold = DataLoader(train_subset, batch_size=64, shuffle=True)
        val_loader_fold = DataLoader(val_subset, batch_size=64, shuffle=False)
```

## 2、带交叉验证思想的网格搜索验证（2）



- 构建神经网络模型、优化器、损失函数，做训练集上做模型训练。

```
# 创建模型
model_cv = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, params['hidden_size']),
    nn.ReLU(),
    nn.Linear(params['hidden_size'], 10)
).to(device)

# 选择优化器
if params['optimizer'] == 'Adam':
    optimizer_cv = optim.Adam(model_cv.parameters(), lr=params['lr'])
else:
    optimizer_cv = optim.Adagrad(model_cv.parameters(), lr=params['lr'])

# 快速训练 (5个epoch)
model_cv.train()
for epoch in range(5):
    for data, target in train_loader_fold:
        data, target = data.to(device), target.to(device)
        optimizer_cv.zero_grad()
        output = model_cv(data)
        loss = nn.CrossEntropyLoss()(output, target)
        loss.backward()
        optimizer_cv.step()
```

- 在验证集上评估模型的准确率，找出最好的参数组合。

```
# 评估
model_cv.eval()
correct = 0
total = 0
with torch.no_grad():
    for data, target in val_loader_fold:
        data, target = data.to(device), target.to(device)
        output = model_cv(data)
        pred = output.argmax(dim=1)
        correct += (pred == target).sum().item()
        total += target.size(0)

score = 100. * correct / total
scores.append(score)
print(f"第 {fold+1} 折准确率: {score:.2f}%")

mean_score = np.mean(scores)
print(f"平均准确率: {mean_score:.2f}%")

if mean_score > best_score:
    best_score = mean_score
    best_params = params
    print(f"新的最佳参数!")

print(f"\n最佳参数: {best_params}")
print(f"最佳交叉验证准确率: {best_score:.2f}%")
```

## 2、带交叉验证思想的网格搜索验证（3）



- 运行结果:

开始不同优化器的交叉验证网格搜索...

测试参数: {'optimizer': 'Adam', 'lr': 0.001, 'hidden\_size': 128}

第1折准确率: 97.17%

第2折准确率: 96.22%

第3折准确率: 97.04%

平均准确率: 96.81%

新的最佳参数!

测试参数: {'optimizer': 'Adam', 'lr': 0.01, 'hidden\_size': 128}

第1折准确率: 94.22%

第2折准确率: 95.64%

第3折准确率: 95.13%

平均准确率: 95.00%

测试参数: {'optimizer': 'Adagrad', 'lr': 0.1, 'hidden\_size': 128}

第1折准确率: 96.16%

第2折准确率: 96.34%

第3折准确率: 95.73%

平均准确率: 96.08%

测试参数: {'optimizer': 'Adagrad', 'lr': 0.01, 'hidden\_size': 256}

第1折准确率: 97.38%

第2折准确率: 97.23%

第3折准确率: 96.97%

平均准确率: 97.19%

新的最佳参数!

最佳参数: {'optimizer': 'Adagrad', 'lr': 0.01, 'hidden\_size': 256}

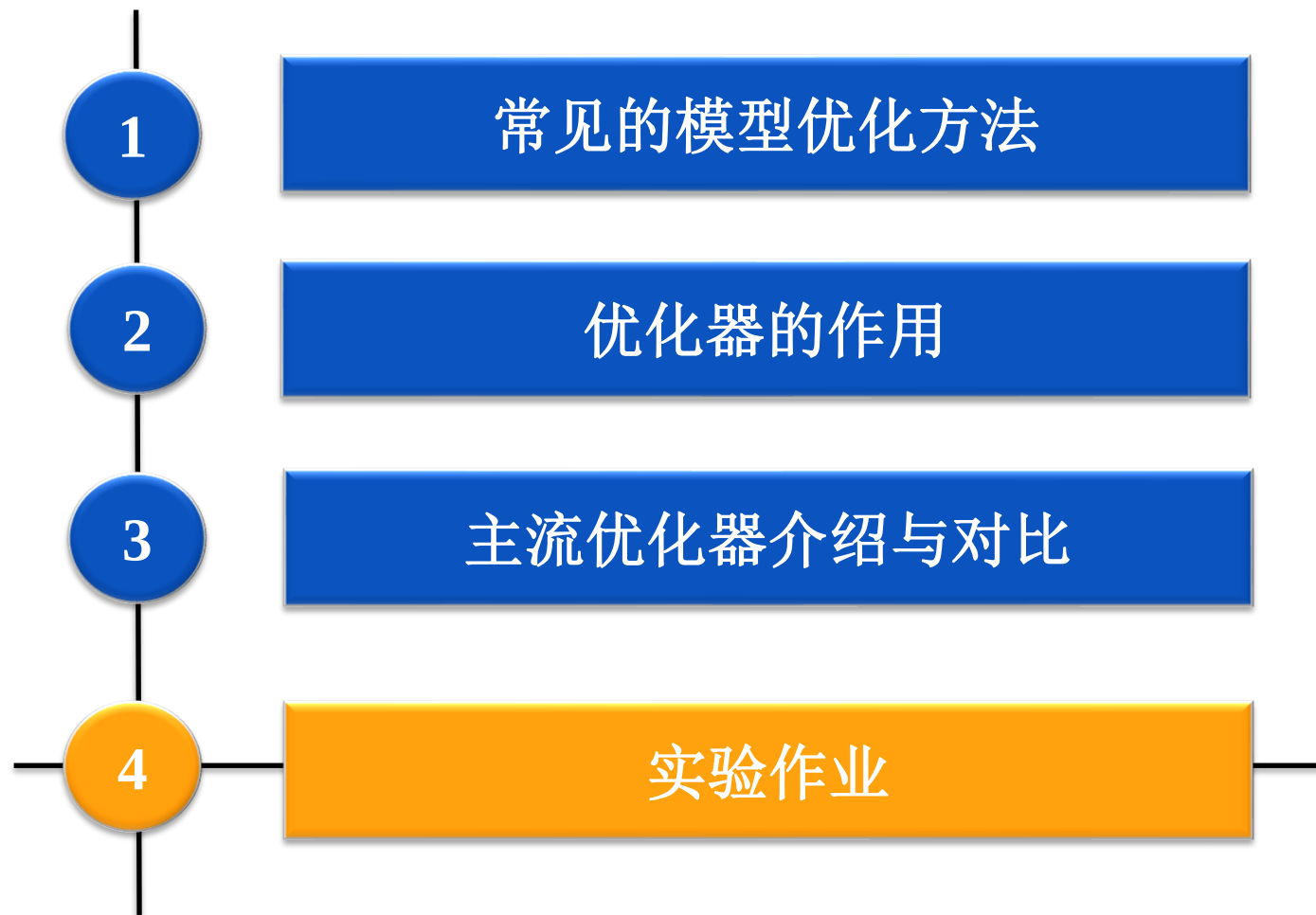
最佳交叉验证准确率: 97.19%

模型训练完成!

- ① 从结果来看，模型在验证集上有97%的准确率，证明是有预测效果的。
  - ② 如果验证集上的准确率过低，则需要考虑其他的模型调优方法，来提升交叉验证的评估结果。
  - ③ 参数组合越多，执行越耗时，特别是在CPU资源上，不宜过多。
- 根据选出的**最优参数组合**，重新**构建模型**，再跑一遍模型训练，并对测试集上做评估。

# 目录

---



# 实验作业——内容

对实验二给出的猫狗数据做分类，完成以下三个任务。

➤ 任务一：基础优化器对比实验，要求：

- 1.在猫狗数据集上分别使用**三种不同的优化器**；
- 2.对每种优化器尝试**3个不同的学习率**，记录每种组合下的：  
训练集/验证集准确率曲线、Loss下降曲线、训练时间对比
- 3.分析不同优化器在**收敛速度、稳定性**方面的差异。



猫狗数据集，其中test测试集有 12500 张，train训练数据有 25000 张图片，猫狗图片各占二分之一。



# 实验作业——内容

## ➤ 任务二：系统化的超参数调优，要求：

- 1.使用网格搜索思想对以下**参数组合**进行调优；
- 2.实现**K折交叉验证**（K=5）来评估每种参数组合；
- 3.找出**最佳参数组合**并在测试集上验证效果；
- 4.分析哪些参数对模型性能影响最大。

```
param_grid = {  
    # 优化相关  
    'optimizer': ['Adam', 'SGD',  
                  'RMSProp'],  
    'lr': [0.001, 0.01, 0.1],  
    'batch_size': [32, 64],  
  
    # 模型结构相关  
    'hidden_size': [128, 256],  
    'dropout_rate': [0.5, 0.7],  
    'activation': ['ReLU', 'sigmoid']  
  
    #也可以补充其他参数组合  
}
```

## ➤ 任务三：模型可视化与错误分析，要求：

- 1.随机选择**5张测试集图片**，显示原图+预测结果；
- 2.绘制**混淆矩阵**，展示出模型在猫/狗分类上的错误数据；
- 3.找出**最难分类的图片**（预测概率接近0.5的样本）并分析原因。

## 实验作业——提交方式

---

实验报告提交至平台 <https://grader.hitsz.edu.cn/courses>

注意：

- 1、用户名用**统一身份登录**；
- 2、请提交到相应的条目「2026春-深度学习」课程 - 实验三；
- 3、提交截止时间：**下周二 6月9号 晚24点**；
- 4、文件夹&压缩包命名要求：**学号\_姓名\_深度学习实验三**
- 5、提交内容：**实验报告**(.word文件)+**代码**(.py/ipynb文件，重要代码处加注释)，一起打包为**zip格式**压缩包。